



IRPF Processor – Visão Geral da Arquitetura

Versão: 2.2

Data: 2026-01-16

Status: 60% Implementado



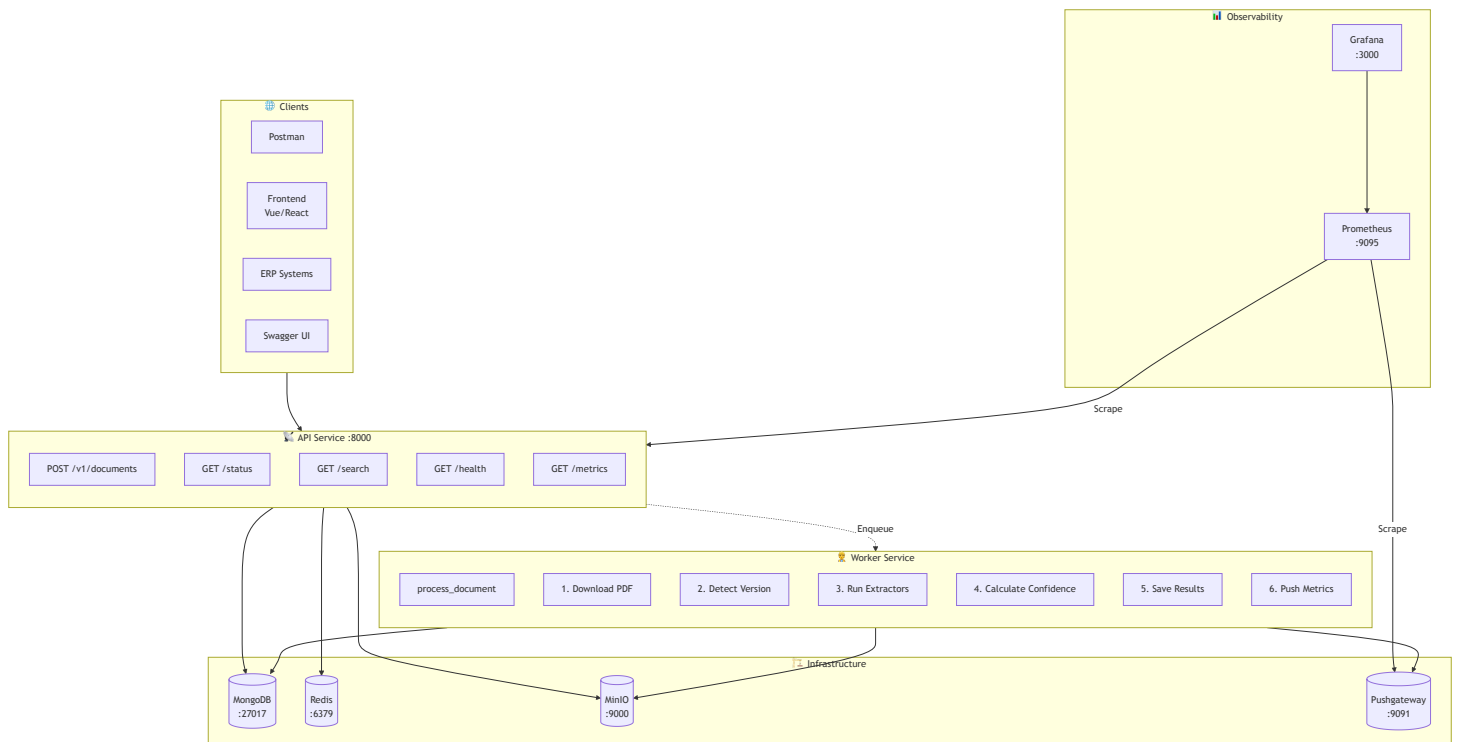
Métricas do Projeto

Categoria	Métrica	Valor
 Código	Linhas de código	~9,500
 Código	Arquivos Python	~80
 Código	Templates YAML	3
 Testes	Testes unitários	366
 Testes	Arquivos de teste	24
 Testes	Cobertura alvo	>80%
 Extração	Extratores de seção	10
 Extração	Versões suportadas	3
 Extração	Campos extraídos	50+
 Massa de Testes	PDFs sintéticos	33
 Massa de Testes	Perfis de teste	9
 Massa de Testes	Taxa de sucesso	71%

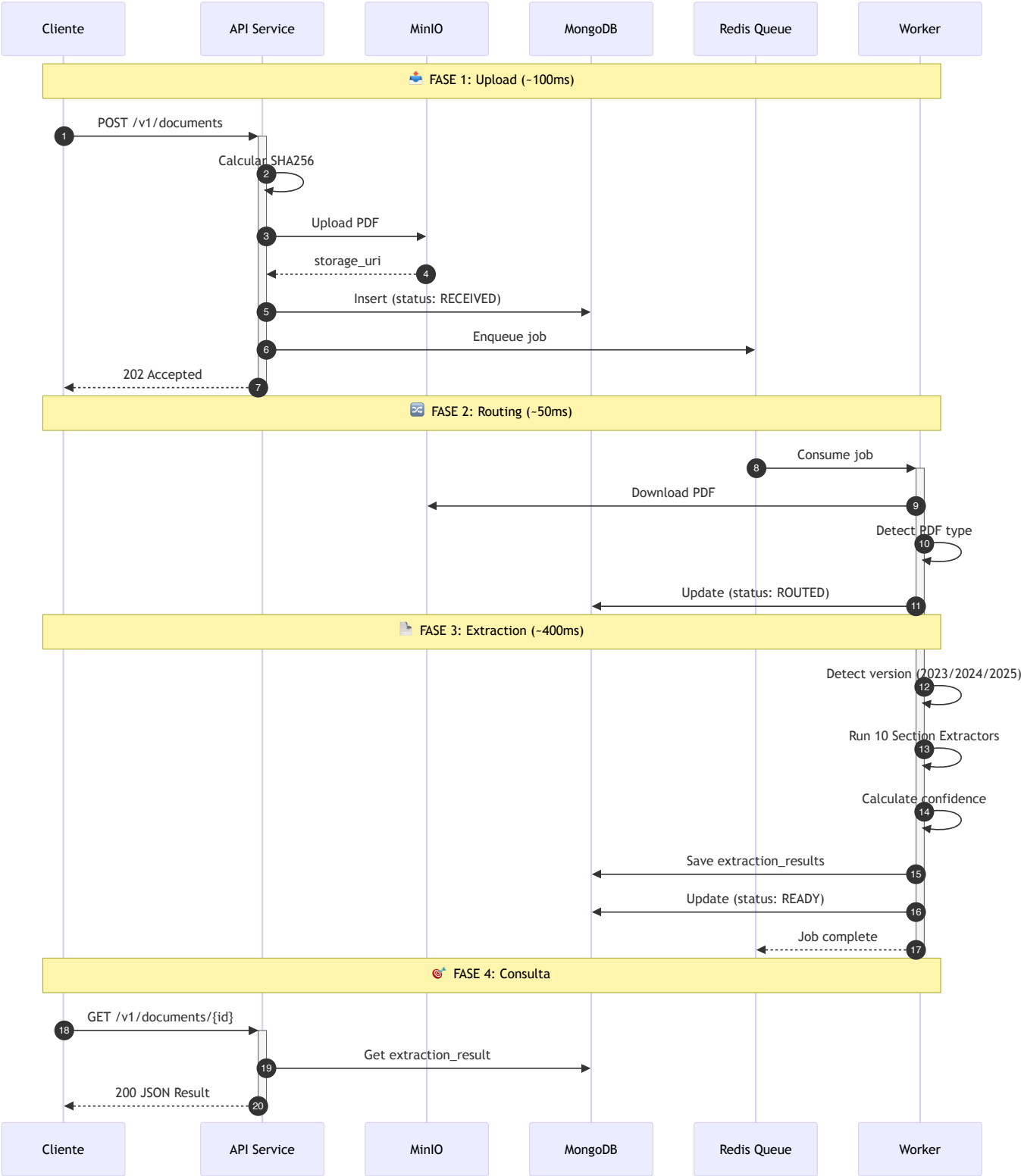
Dashboard de Status

Feature	Status
API REST (FastAPI)	✅ Implementado
Upload de Documentos	✅ Implementado
Worker Dramatiq	✅ Implementado
MongoDB + Redis + MinIO	✅ Implementado
Prometheus + Grafana	✅ Implementado
Parser IRPF (10 extratores)	✅ Implementado
Detecção de Versão (2023-2025)	✅ Implementado
Sistema de Templates YAML	✅ Implementado
Gerador de Massa de Testes	✅ Implementado
Search API (CPF, Ano, Cidade)	✅ Implementado
SSE Events	⌚ Pendente
OCR para PDFs escaneados	⌚ Pendente

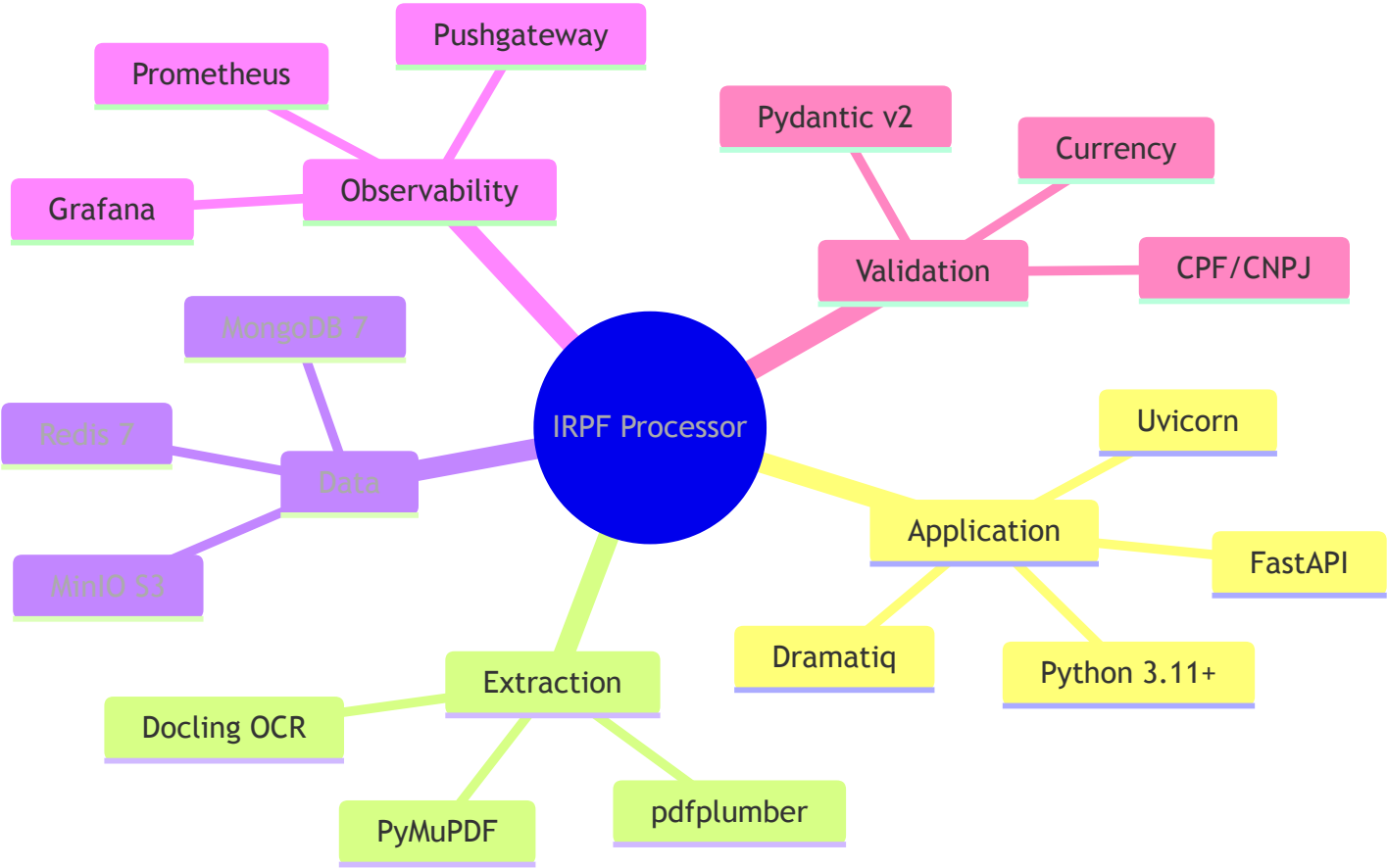
1. Arquitetura de Alto Nível



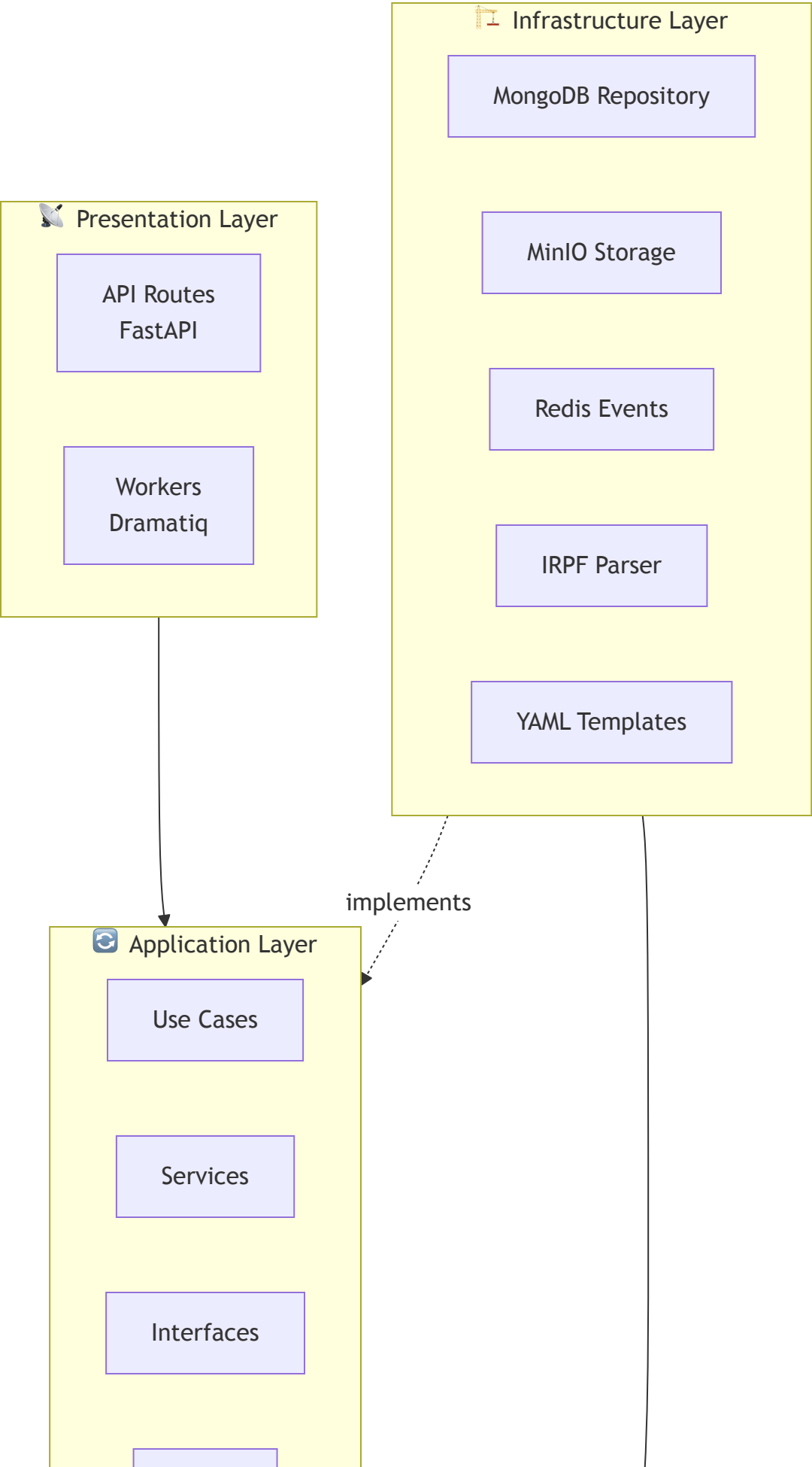
2. Fluxo de Processamento de Documento

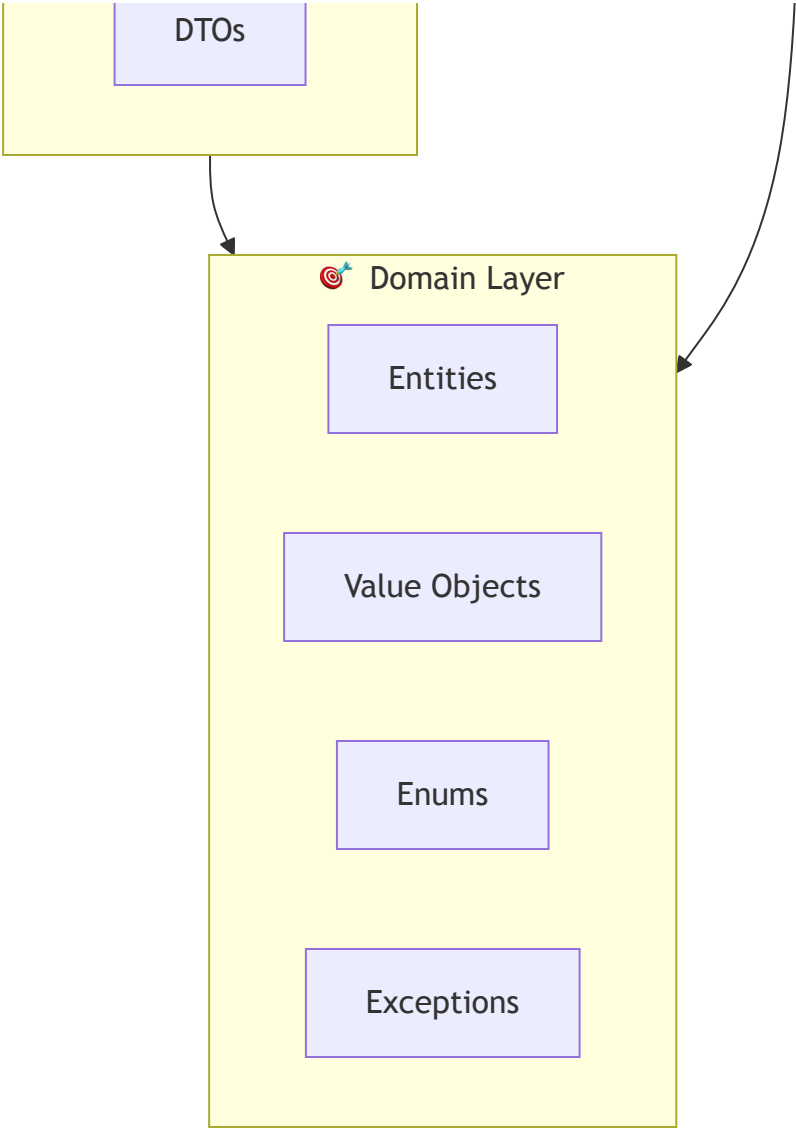


3. Stack Tecnológica



4. Arquitetura de Camadas (Clean Architecture)



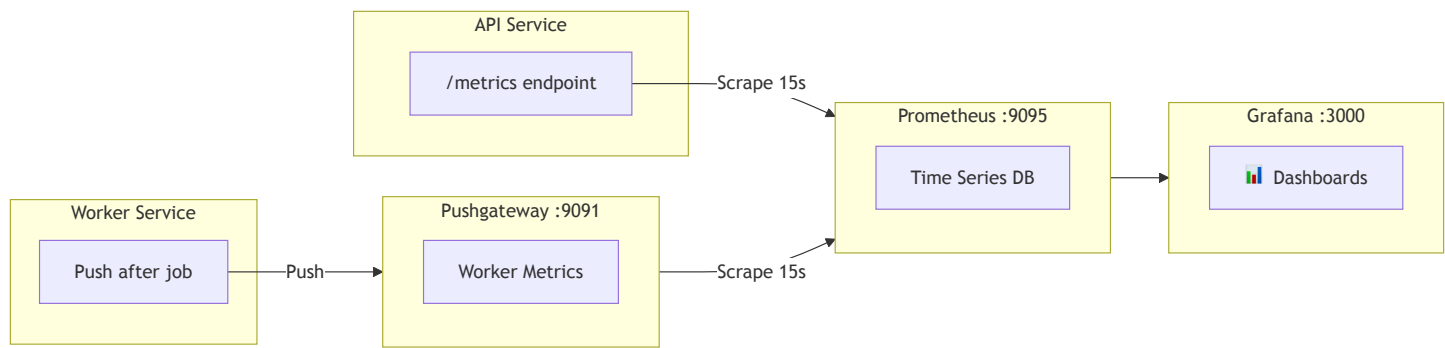


5. Estrutura de Pastas

```
src/irpf_processor/
├── main.py                # FastAPI entrypoint
├── config.py              # Pydantic Settings
├──
├── domain/                # 🎯 DOMAIN LAYER
│   ├── entities/document.py
│   ├── enums/document_status.py
│   ├── enums/pdf_type.py
│   ├── exceptions/domain_exceptions.py
│   └── value_objects/extraction_result.py
├──
├── application/           # 🔄 APPLICATION LAYER
│   ├── interfaces/repositories.py
│   ├── interfaces/event_publisher.py
│   ├── services/document_service.py
│   └── dto/responses.py
├──
├── infrastructure/        # 🏗️ INFRASTRUCTURE LAYER
│   ├── persistence/document_repository.py
│   ├── storage/minio_storage.py
│   ├── extraction/
│   │   ├── irpf_parser.py        # Facade
│   │   ├── version_detector.py
│   │   ├── text_extractor.py
│   │   ├── table_extractor.py
│   │   └── extractors/           # 10 Strategy extractors
│   └── receita_federal/
│       └── test_pdf_generator.py
├──
├── presentation/          # 🖥️ PRESENTATION LAYER
│   ├── api/routes/
│   └── workers/extraction_worker.py
├──
├── templates/definitions/ # 📄 YAML TEMPLATES
│   ├── irpf_2023.yaml
│   ├── irpf_2024.yaml
│   └── irpf_2025.yaml
├──
└── cli/                   # 🛠️ CLI TOOLS
```

- └ generate_test_models.py
- └ sync_layouts.py

6. Sistema de Métricas



Métricas Disponíveis

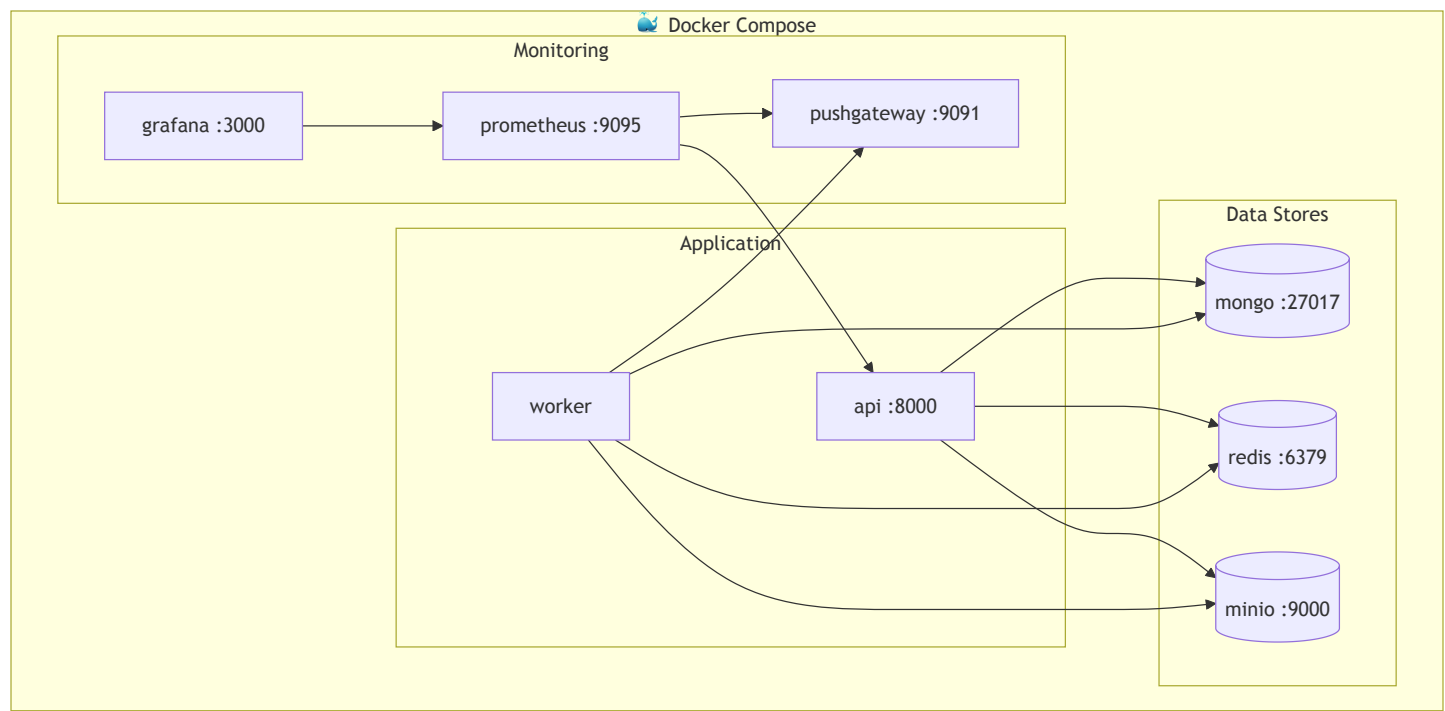
Métrica	Tipo	Labels
irpf_documents_uploaded_total	Counter	tenant_id
irpf_documents_processed_total	Counter	tenant_id, status, pdf_type
irpf_extraction_duration_seconds	Histogram	tenant_id, pdf_type, version
irpf_section_extraction_total	Counter	section, version, success
irpf_extraction_warnings_total	Counter	tenant_id, warning_type
irpf_failures_total	Counter	tenant_id, stage, error_type

7. API Endpoints

Método	Endpoint	Descrição	Status
GET	/health	Liveness check	✓
GET	/ready	Readiness check	✓

Método	Endpoint	Descrição	Status
GET	/metrics	Prometheus metrics	✓
POST	/v1/documents	Upload PDF	✓
GET	/v1/documents/{id}	Get result	✓
GET	/v1/documents/{id}/status	Get status	✓
GET	/v1/documents/{id}/events	SSE stream	🕒
GET	/v1/search	Search documents	✓
GET	/v1/search/by-cpf/{cpf}	Search by CPF	✓
GET	/v1/search/stats	Statistics	✓

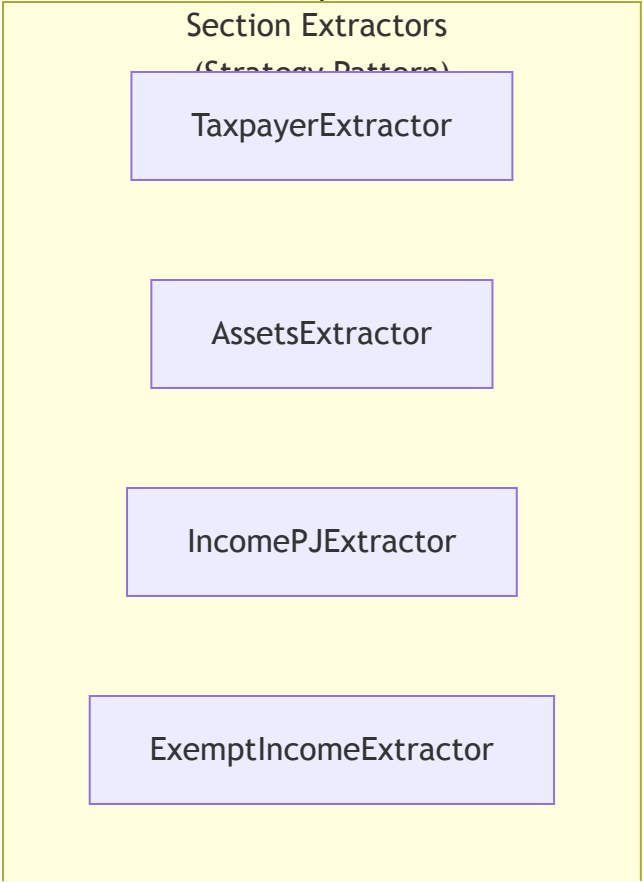
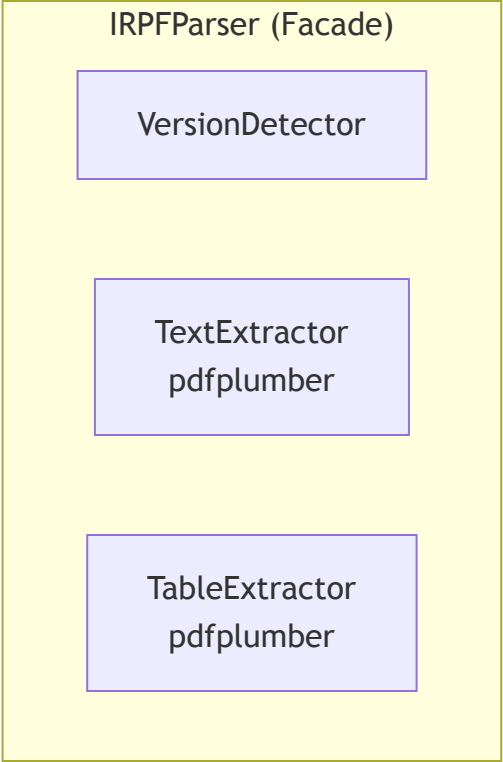
8. Docker Compose Services



9. Sistema de Parser IRPF

O coração da aplicação é o **IRPF Parser**, uma solução flexível e extensível.

9.1 Arquitetura do Parser



ExclusiveIncomeExtractor

RuralPropertiesExtractor

RuralIncomeExtractor

RuralResultsExtractor

RuralAssetsExtractor

RuralDebtsExtractor

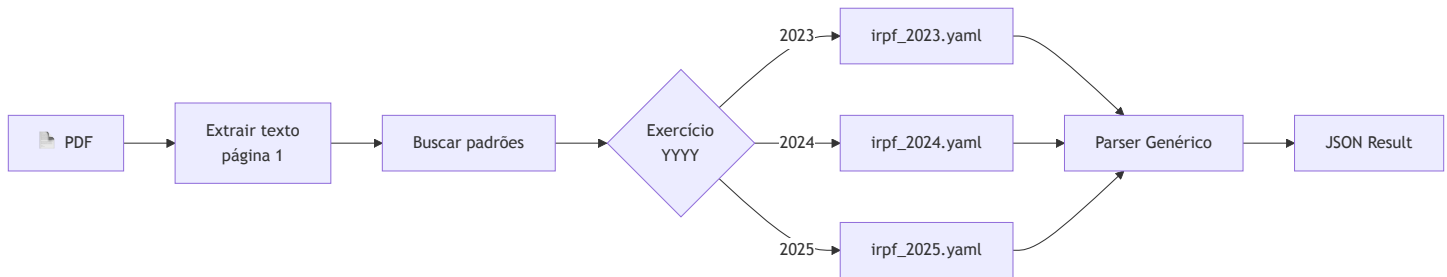
Output

IRPFDeclarationResult

Confidence: 0.0 - 1.0

Warnings: list

9.2 Detecção Automática de Versão



9.3 Sistema de Templates YAML

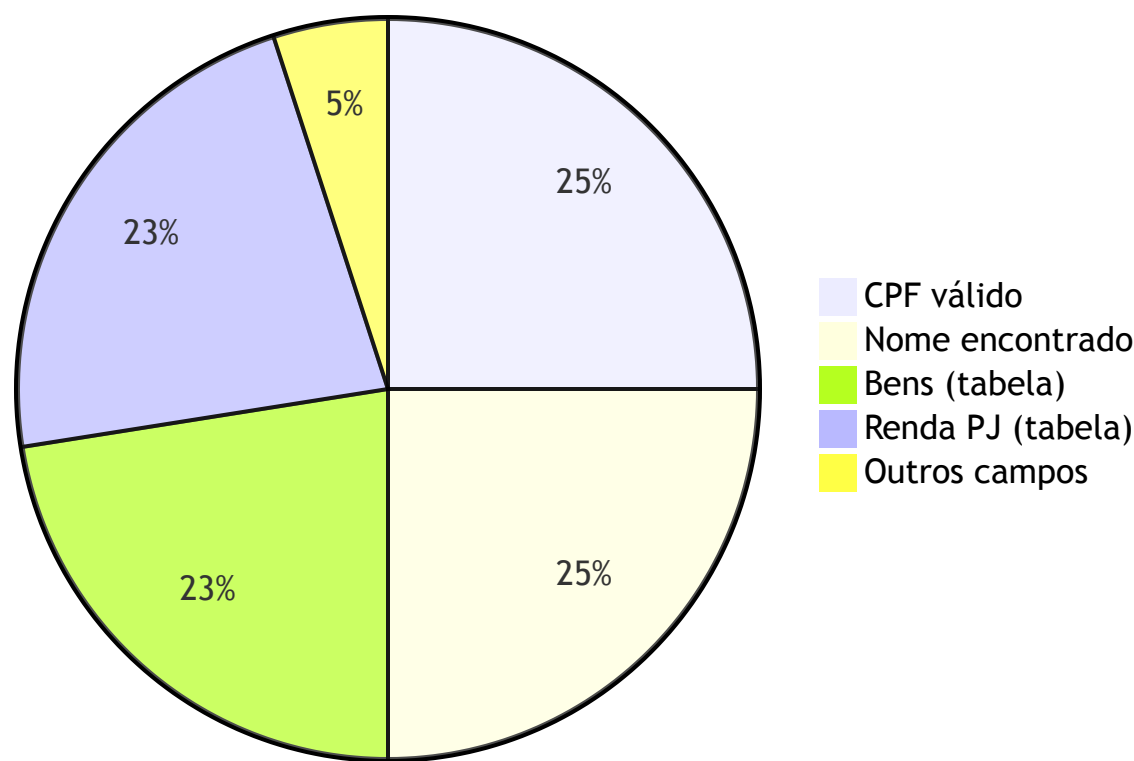
```
# templates/definitions/irpf_2025.yaml
metadata:
  version: "2025"
  exercise_year: "2025"
  calendar_year: "2024"

sections:
  taxpayer_identification:
    name: "Identificação do Contribuinte"
    required: true
    fields:
      - { name: cpf, type: cpf, required: true }
      - { name: name, type: string, required: true }

  assets_declaration:
    name: "Bens e Direitos"
    repeatable: true
    has_totals: true
```

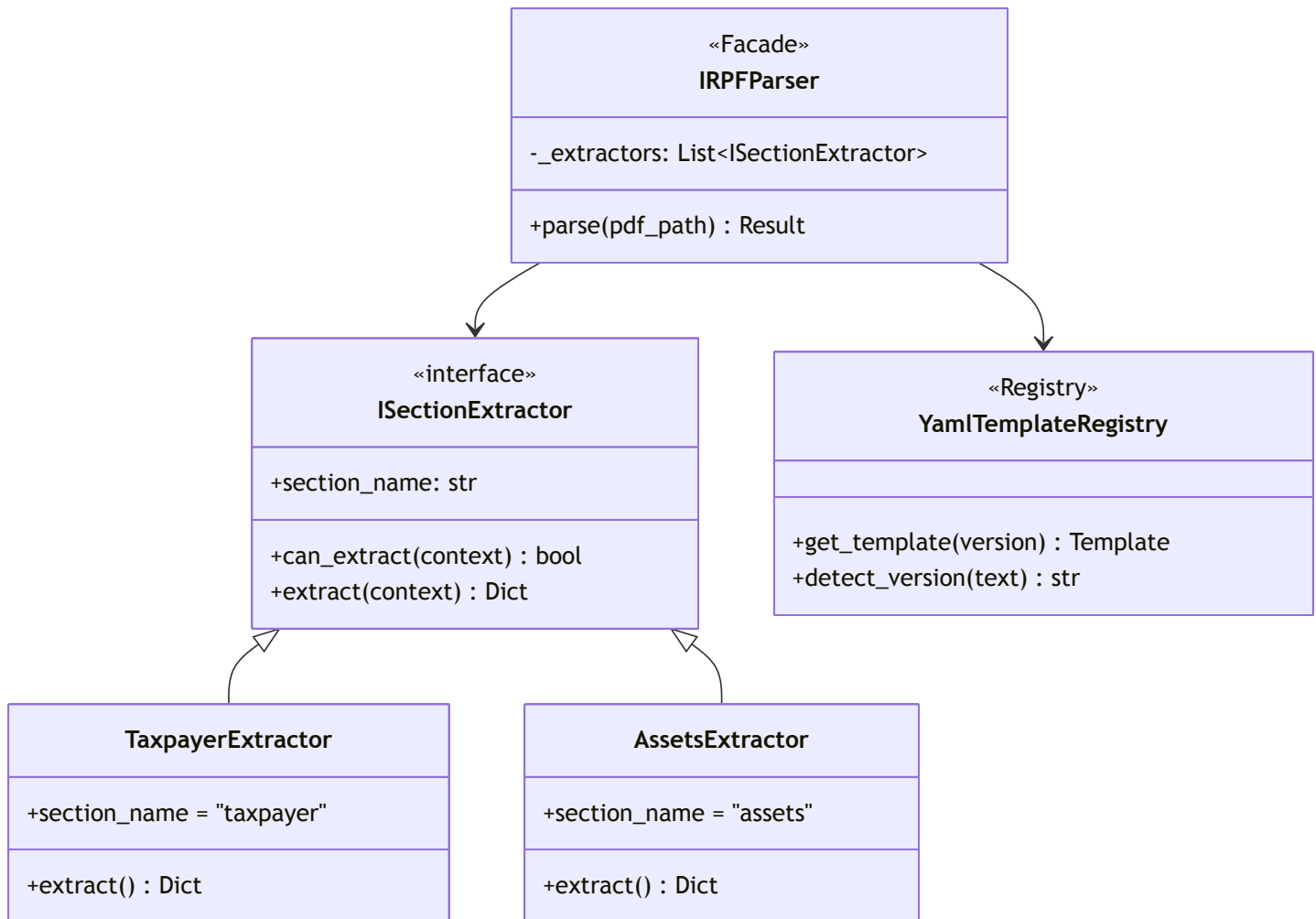

9.4 Cálculo de Confiança

Confidence Score Weights



Campo	Peso	Confiança Típica
CPF encontrado e válido	1.0	100%
Nome do contribuinte	1.0	100%
Bens e Direitos (tabela)	0.9	90%
Rendimentos PJ (tabela)	0.9	90%
Totais calculados	0.95	95%

9.5 Design Patterns Aplicados



10. 🛠️ Ferramentas CLI

Gerador de Massa de Testes

Gerar 50 declarações aleatórias

```
python -m irpf_processor.cli.generate_test_models --random 50
```

Gerar perfis específicos

```
python -m irpf_processor.cli.generate_test_models --profiles wealthy rural_large
```

Gerar e testar automaticamente

```
python -m irpf_processor.cli.generate_test_models --test --max-tests 20
```

Perfis de Declaração

Perfil	Bens	Fontes	Rural	Descrição
minimal	1-3	1	✗	Declaração mínima
simple	3-8	1-2	✗	Contribuinte simples
average	5-15	1-3	✗	Brasileiro médio
wealthy	15-40	2-5	✗	Alto patrimônio
ultra_rich	40-100	5-10	✓	Ultra rico
rural_small	3-10	1-2	✓	Produtor rural pequeno
rural_large	10-30	2-4	✓	Produtor rural grande
retired	5-20	0-1	✗	Aposentado
investor	20-50	1-2	✗	Investidor

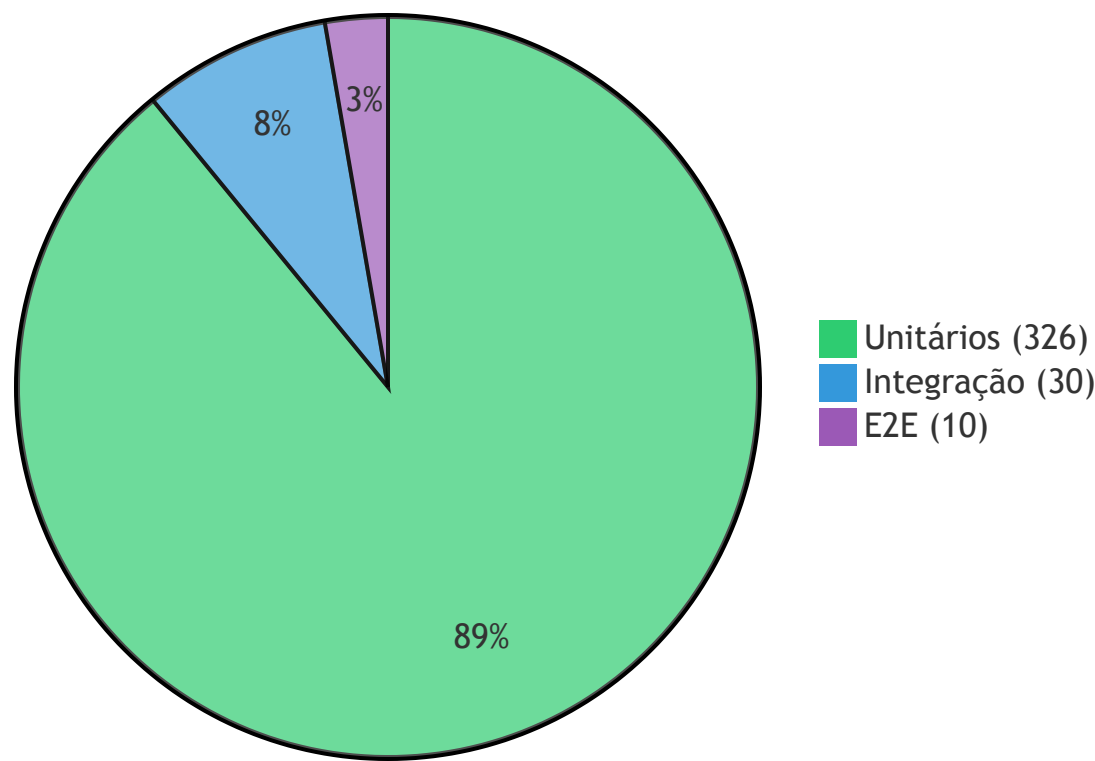
Dados Sintéticos

Dado	Quantidade	Validação
CPFs	Gerados	Dígitos verificadores válidos
CNPJs	20 empresas	Dígitos verificadores válidos
Nomes	64	32 masculinos + 32 femininos
Sobrenomes	40	Sobrenomes brasileiros
Cidades	24	Com UF e CEP corretos
Bancos	10	CNPJs válidos
Tipos de Bens	28	Códigos da Receita Federal

11. 🧪 Estratégia de Testes

Pirâmide de Testes

Distribuição de Testes



Cobertura por Componente

Componente	Testes	Cobertura
Domain (Entities, Enums)	45	~95%
Field Extractors (CPF, CNPJ, Currency)	62	~90%
Section Extractors	85	~85%
Version Detector	28	~90%
Template Registry	18	~80%
API Routes	42	~85%
Repositories	35	~80%

Componente	Testes	Cobertura
Services	51	~85%

Executar Testes

```
# Todos os testes
pytest tests/ -v
```

```
# Com cobertura
pytest tests/ --cov=src/irpf_processor --cov-report=html
```

```
# Teste em massa do parser
python -m irpf_processor.cli.generate_test_models --test --max-tests 50
```

12. Decisões Arquiteturais (ADRs)

ADR-001: MongoDB como Document Store

Aspecto	Detalhe
Contexto	Armazenar declarações IRPF com estrutura variável por ano
Decisão	Usar MongoDB
Justificativa	Schema flexível, documentos JSON, motor async, queries ricas

ADR-002: Parser Baseado em Templates YAML

Aspecto	Detalhe
Contexto	Formato IRPF muda anualmente
Decisão	Templates YAML com detecção automática
Justificativa	Novo ano = criar YAML (sem código), declarativo, versionável

ADR-003: Strategy Pattern para Extratores

Aspecto	Detalhe
Contexto	Cada seção tem lógica diferente
Decisão	Interface <code>ISectionExtractor</code> com 10 implementações
Justificativa	SRP, Open/Closed, testabilidade, flexibilidade

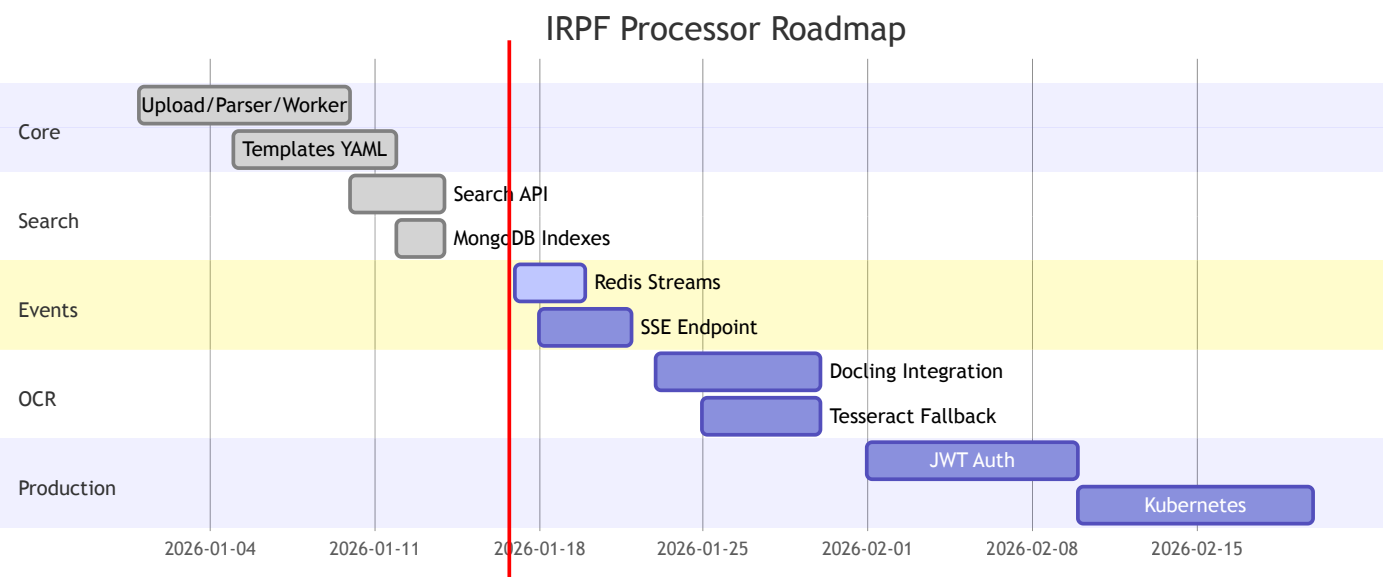
ADR-004: Worker Síncrono

Aspecto	Detalhe
Contexto	<code>asyncio.run()</code> causava <code>Event loop is closed</code>
Decisão	Usar <code>pymongo sync</code> no worker
Justificativa	<code>Dramatiq</code> usa threads, estabilidade, ~0.5s por doc

ADR-005: Pushgateway para Métricas

Aspecto	Detalhe
Contexto	Workers são efêmeros
Decisão	Push para Pushgateway após cada job
Justificativa	Métricas persistem, scrape unificado

13. Roadmap de Features



14. Links Úteis

Recurso	URL	Descrição
API	http://localhost:8000	FastAPI REST API
Swagger	http://localhost:8000/docs	Documentação interativa
ReDoc	http://localhost:8000/redoc	Documentação alternativa
Grafana	http://localhost:3000	Dashboards (admin/admin)
Prometheus	http://localhost:9095	Queries de métricas
MinIO	http://localhost:9001	Object storage
Pushgateway	http://localhost:9091	Worker metrics

15. Comandos Essenciais

```
# Subir infraestrutura
docker compose up -d

# Ver logs
docker compose logs -f api worker

# Testar health
curl http://localhost:8000/health

# Upload documento
curl -X POST http://localhost:8000/v1/documents \
  -H "X-Tenant-ID: test" \
  -F "file=@documento.pdf"

# Buscar por CPF
curl "http://localhost:8000/v1/search/by-cpf/123.456.789-00" \
  -H "X-Tenant-ID: test"

# Gerar massa de testes
python -m irpf_processor.cli.generate_test_models --random 50 --test
```

Feito por Felipe Scaphe com ❤️ para o AsaBank

Janeiro 2026

 Estatísticas Finais

Métrica	Valor
Linhas de código	~9,500
Testes automatizados	366
Extratores de seção	10
Templates YAML	3 (2023, 2024, 2025)

Métrica	Valor
PDFs de teste	33
Taxa de sucesso	71%
Tempo médio de processamento	0.5s